JUEGALMI — ESPECIFICACIÓN DE ARQUITECTURA

Propuesta Técnica, Modelo de Datos y Documentación del Proyecto (Reto 2)

CENTRO:	CURSO / CICLO:	ARQUITECTURA:	FECHA:
Centro Estudios ALMI	**2025-2026 / DAM 2**	**Android + Ionic + Symfony**	**Octubre 2026**

Propósito del Proyecto y Visión Comercial

JuegAlmi da respuesta a las necesidades operativas de la tienda mediante una solución multiplataforma integrada. A diferencia de distribuidores puramente digitales, la app combina la venta/alquiler de videojuegos con la venta y servicio técnico presencial de consolas, móviles y tablets, manteniendo la fidelización del cliente local mediante geolocalización activa.

1. STACK TECNOLÓGICO Y ESTRATEGIA MULTIPLATAFORMA

Cliente Android Nativo

GOOGLE PLAY UI

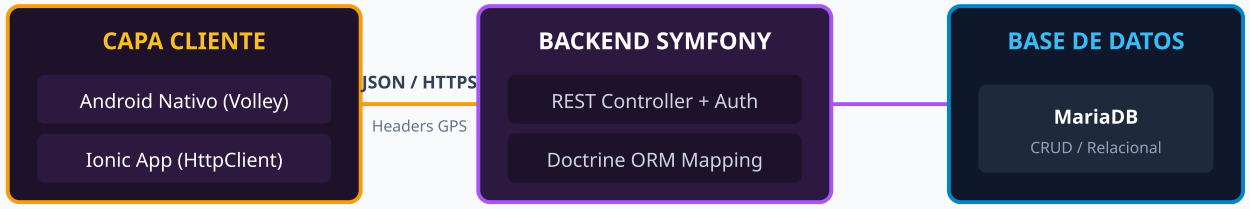
Desarrollado en Java/Kotlin con arquitectura basada en `RecyclerView`, `CardView`, `Tabs` y `Navigation Drawer`. Consumo de API REST vía **Volley / Retrofit** y carga eficiente de imágenes mediante **Glide/Picasso**.

Cliente Híbrido (iOS / Win)

IONIC / ANGULAR

Desarrollo híbrido garantizando uniformidad gráfica exacta. Gestión de llamadas asíncronas con promesas/factories y almacenamiento en `LocalStorage`/`SessionStorage` para mantenimiento de sesión.

2. DIAGRAMA DE ARQUITECTURA DE CAPAS

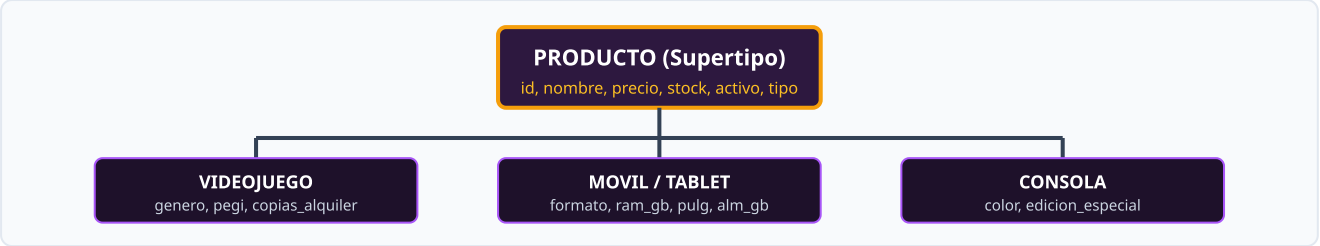


3. PUNTOS CLAVE INNEGOCIABLES DEL NEGOCIO

- **Geolocalización en Tiempo Real:** Envío obligatorio de cabeceras GPS en todas las peticiones para trazabilidad operativa del usuario.
- **Gestión de Eficiencia:** Optimización de memoria RAM mediante carga progresiva e imágenes en caché local (`Glide`/`Picasso`).
- **Identidad Homogénea:** Apariencia visual idéntica entre plataformas emulando la interfaz fluida de Google Play.

4. MODELO RELACIONAL I: SUPERTIPO PRODUCTO Y VENTAS

El catálogo utiliza una jerarquía de herencia **total y disjunta** sobre la tabla supertipo `producto`, permitiendo gestionar videojuegos, teléfonos y consolas dentro de un único carrito unificado.



Especificación de Tablas de Catálogo y Ventas

TABLA	COLUMNAS CLAVE	RESTRICCIONES Y REGLAS DE NEGOCIO
producto	`id`, `nombre`, `precio`, `stock`, `activo`, `tipo`, `marca_id`	Baja lógica mediante `activo = 0`. Discriminador gestionado por Doctrine. `precio` ≥ 0.
videojuego	`id` (FK), `genero`, `pegi`, `precio_alquiler`, `copias_alquiler`, `plataforma_id`	Gestiona alquileres y ventas. Copias de alquiler independientes del stock de venta. FK → `plataforma`.
movil	`id` (FK), `formato`, `pantalla_pulgadas`, `ram_gb`, `almacenamiento_gb`	`formato` IN ('movil', 'tablet'). Renombrado para evitar conflictos con discriminador supertipo.
marca / plataforma	`id`, `nombre` (UNIQUE)	Catálogos auxiliares normalizados para evitar duplicidad en filtros de búsqueda.

5. ESTRUCTURA DE COMPRAS Y PEDIDOS

TABLA	COLUMNAS	DESCRIPCIÓN Y COMPORTAMIENTO
pedido	`id`, `fecha`, `estado`, `metodo_pago`, `latitud`, `longitud`, `usuario_id`	Estados: `pendiente` → `preparado` → `entregado` / `cancelado`. Almacena coordenadas GPS obligatorias del cliente.
linea_pedido	`id`, `pedido_id`, `producto_id`, `cantidad`, `precio_unitario`	Relación N:M. Congela el precio del producto en la fecha de compra. UNIQUE `(pedido_id, producto_id)`.

6. GESTIÓN DE USUARIOS, SESIONES Y SEGURIDAD

Acceso centralizado con tres roles (`cliente`, `tecnico`, `admin`). La autenticación utiliza tokens persistentes de 64 caracteres SHA-256 almacenados de forma segura.

TABLA	COLUMNAS CLAVE	REGLAS DE NEGOCIO E INTEGRIDAD
usuario	`id`, `nombre`, `email` (UQ), `password`, `dni` (UQ), `rol`, `foto`, `departamento_id`	Contraseña en hash. `rol` ∈ {'cliente', 'tecnico', 'admin'}. CHECK: Si `rol = 'tecnico'` ⇒ `departamento_id` NO es nulo.
token	`id`, `token_hash` (UQ), `fecha_creacion`, `fecha_expira`, `usuario_id`	Token en claro enviado solo en login. La BBDD guarda el SHA-256. Expiración a las 24h. Borrado en CASCADE.

7. SISTEMA DE SOPORTE TÉCNICO Y TALLER

División en **dos departamentos fijos**: Telefonía (smartphones/tablets) y Consolas. Las reparaciones siguen un ciclo de vida con asignación técnica y presupuesto.

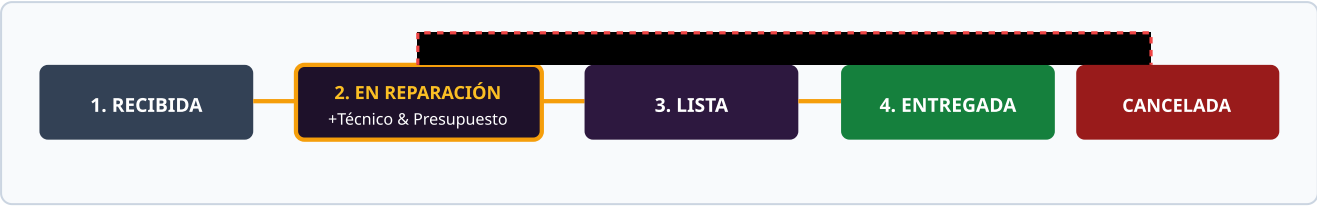


TABLA	COLUMNAS	DESCRIPCIÓN E INTEGRIDAD
departamento	`id`, `nombre` (UNIQUE)	Valores fijos: 'Telefonía' y 'Consolas'. Asociado a técnicos y servicios.
servicio	`id`, `nombre`, `descripcion`, `precio_orientativo`, `departamento_id`	Contenido dinámico para las pestañas de la sección "Servicios" de la app.
reparacion	`id`, `dispositivo`, `problema`, `estado`, `presupuesto`, `foto`, `cliente_id`, `tecnico_id`, `departamento_id`	Doble FK a `usuario` (`cliente_id` y `tecnico_id`). Captura de foto de la avería mediante cámara. Coordenadas GPS obligatorias.

8. AUDITORÍA GPS Y VISTA DE TIENDA ESTILO GOOGLE PLAY

Tabla `registro_ubicacion` INNEGOCIABLE

Registra latitud, longitud, endpoint y método HTTP en **cada consulta a la API** enviados mediante las cabeceras custom `X-Latitud` y `X-Longitud`. Datos protegidos bajo RGPD.

Tablas `tienda` / `foto_tienda` GOOGLE PLAY UI

Soportan las opciones nativas de la vista principal: Servicios, Ubicación en mapa, Galería de fotos con carga diferida/placeholders y la Historia de la tienda.

9. ESPECIFICACIÓN DE ENDPOINTS PRINCIPALES (API REST)

MÉTODO	ENDPOINT API	UBICACIÓN GPS	DESCRIPCIÓN Y RESPUESTA
POST	`/api/login`	Opcional	Autentica credenciales y genera un nuevo Token de 24h.
GET	`/api/videojuegos`	Obligatoria	Listado filtrado por plataforma/género. Registra ubicación.
POST	`/api/pedidos`	Obligatoria	Crea pedido en transacción única. Descuenta stock.
POST	`/api/reparaciones`	Obligatoria	Registra avería, foto de cámara y asigna departamento.
GET	`/api/tienda/galeria`	Obligatoria	Devuelve las fotos de la tienda para carga asíncrona.

10. FLUJO DE AUTENTICACIÓN E INTERCEPCIÓN DE PETICIONES

Mecanismo Transversal de Seguridad e Intercepción HTTP

Todas las peticiones salientes desde Android (`Retrofit`/`Volley`) e Ionic (`HttpClient Interceptor`) adjuntan automáticamente:

- `Authorization: Bearer ` para validación de sesión.
- `X-Latitud` y `X-Longitud` leídas del sensor GPS del dispositivo.

11. REQUISITOS NO FUNCIONALES (RNF) Y RENDIMIENTO

Carga Asíncrona Eficiente

Uso de hilos secundarios (`AsyncTask` / Corrutinas en Android; Promesas/Factories en Ionic) para evitar el bloqueo del hilo principal de la interfaz.

Gestión de Memoria e Imágenes

Integración de **Glide / Picasso** para reescalado automático, caché de imágenes y placeholders en caso de retardo en la red.

Optimización de CPU

Consumo mínimo de microprocesador garantizando navegación fluida y rápida respuesta al toque del usuario.

Resiliencia de Datos

Validación estricta de formularios e intenciones. Persistencia CRUD mediante Servicios Web RESTful.

12. ALINEACIÓN CON LA RÚBRICA DE EVALUACIÓN

MÓDULO / COMPETENCIA	CRITERIO DE EXCELENCIA (PUNTUACIÓN MÁXIMA)
Acceso a Datos	Servicio Web RestFull (GET, POST, PUT, DELETE) con ORM Symfony e integración de Tokens de seguridad HTTPS.
Desarrollo de Interfaces	Diseño homogéneo entre plataformas emulando la UI de Google Play mediante `RecyclerView` y `CardView`. Validaciones activas.
Programación Multimedia	Geolocalización funcional, integración de cámara para fotos de reparaciones y perfil, procesos libres de errores y gestión de sesiones.
Servicios y Procesos	Carga asíncrona de datos progresiva, uso eficiente de librerías de red (`Volley`/`Retrofit`) y animación de componentes.

13. CRONOGRAMA DE ENTREGABLES E HITOS

Planificación de Trabajo (Semanas 1 a 11)

- **Hito 1 (Semana 5):** Documento de propuesta final, diagramas E-R, prototipo de diseño visual e interfaces.
- **Hito 2 (Semana 11):** APK funcional para Android, desproliegue en app híbrida, persistencia CRUD completa, GPS activo, dossier y exposición de 20 min con demo.